

JMeter Performance Testing

Setup, Installation, and Run Guide — VIP Connect (VIPC) Transaction Flows

Author: Brock Pivrotto | **Last Updated:** August 2026 | **Product:** VIP Connect / VIP Preferred SDK

1. Overview

This document explains how to set up Apache JMeter, load the VIP Connect (VIPC) transaction flow performance test script, and execute a load test against the VIP Preferred SDK API. The primary script covered is TransactionFlows.jmx, which simulates concurrent patron withdraw and deposit flows through the SDK.

JMeter version in use: **Apache JMeter 5.5**

Script location (OneDrive): `OneDrive - Pavilion Payments\Tools\apache-jmeter-5.5\LoadTestScripts\TransactionFlows.jmx`

Target environment: **QA** — `qa.api-gaming.paviliononline.io`

2. Prerequisites

Before running JMeter tests, confirm the following are in place:

Requirement	Details
Java 8 or later (64-bit)	JMeter requires Java. Verify with: <code>java -version</code> in a terminal.
Apache JMeter 5.5	Already installed. See Section 3 for the install path.
Valid operator token (JWT)	The Bearer token in the script expires. Update it before running (see Section 6).
VPN / network access	The QA environment is internal. Connect to the Pavilion VPN before running tests.
QA environment available	Confirm the QA API is up before starting a load run.

3. JMeter Installation

3.1 Existing Installation

JMeter 5.5 is already installed locally. Two copies exist:

- Local (Documents): `C:\Users\BrockPivrotto\Documents\apache-jmeter-5.5`

- OneDrive (with scripts): `C:\Users\BrockPivirotto\OneDrive - Pavilion Payments\Tools\apache-jmeter-5.5\apache-jmeter-5.5`

Use the local install to run JMeter, and load scripts from the OneDrive LoadTestScripts folder.

3.2 Fresh Installation (if needed)

Follow these steps if setting up JMeter on a new machine:

1. Verify Java is installed. Open a command prompt and run:

```
java -version
```

If Java is missing, download the JDK from <https://adoptium.net> and install it. Set the `JAVA_HOME` environment variable to the JDK install directory.

2. Download Apache JMeter 5.5 from the Apache JMeter website. Choose the .zip binary.
3. Extract the zip to a local folder, for example: `C:\Tools\apache-jmeter-5.5`
4. No further installation is required. JMeter runs directly from the extracted folder.

3.3 Verifying the Installation

Open a command prompt and navigate to the JMeter bin folder, then run:

```
cd C:\Users\BrockPivirotto\Documents\apache-jmeter-5.5\bin
jmeter.bat
```

The JMeter GUI should open. If it does not, check that `JAVA_HOME` is set correctly.

4. Understanding the TransactionFlows.jmx Script

4.1 What the Script Tests

TransactionFlows.jmx is a load test script that simulates two complete VIPC transaction flows running concurrently in every iteration: a full Withdraw flow and a full Deposit flow. Each flow mirrors the exact API call sequence a real casino cage or kiosk application would execute.

Transaction Controller	Steps Executed
Withdraw	1. AccountLookup → 2. Create Existing Session → 3. Withdraw → 4. Approve Withdraw
Deposit	1. AccountLookup → 2. Create Existing Session → 3. Deposit

4.2 Global Variables

These variables are defined at the Test Plan level and can be updated without editing individual requests:

Variable	Default Value	Purpose
host	qa.api-gaming.paviliononline.io	Target API hostname
port	443	HTTPS port
token	(JWT Bearer token)	Operator bearer token — update when expired

4.3 Thread Group Configuration

The Thread Group controls how many virtual users run and for how long:

Setting	Value	Meaning
Number of Threads (Users)	10	10 virtual users run concurrently
Ramp-Up Period	60 seconds	JMeter starts 1 new user every 6 seconds until all 10 are active
Loop Count	100	Each user repeats the full test plan 100 times
Total Requests	~10,000	10 users × 100 loops × ~10 API calls per iteration

NOTE: To run a quick smoke test before a full load run, change Loops to 1 and Threads to 1. Reset to 10/100 for load testing.

4.4 Withdraw Transaction Flow

The Withdraw controller executes these four HTTP requests in sequence for each iteration:

5. **AccountLookup** — POST `/sdk/api/patronsession/accountlookup`. Verifies the patron exists and retrieves their profile. Uses operator Bearer token.
6. **Create Session (Existing)** — POST `/sdk/api/patronsession/existing`. Opens a patron session for the withdraw transaction. The JSR223 PreProcessor generates a random PatronId and TransactionId via Groovy. The sessionId is extracted from the response using a JSONPostProcessor and stored in `_${sessionId}`.

7. **Withdraw** — POST `/sdk/api/funds/withdraw`. Queues the withdrawal. Uses `Authorization: PatronSession` and `Session-Id: ${sessionId}` (patron auth, not operator auth).
8. **ApproveWithdraw** — POST `/sdk/api/operator/WithdrawApproval`. Approves the queued withdrawal from the operator side. Uses operator Bearer token.

4.5 Deposit Transaction Flow

The Deposit controller runs in parallel with Withdraw (same iteration) and executes three steps:

9. **AccountLookup** — POST `/sdk/api/patronsession/accountlookup`. Same patron as Withdraw flow.
10. **Create Session (Existing)** — POST `/sdk/api/patronsession/existing`. Opens a new patron session for the deposit. Uses a separate random TransactionId (stored in `${transactionID}`) and extracts session to `${depositSessionID}`.
11. **Deposit** — POST `/sdk/api/funds/deposit`. Submits the deposit. Uses patron session auth with `${depositSessionID}`.

4.6 Dynamic Data Generation

To prevent transaction ID collisions across concurrent threads, the script uses a Groovy JSR223 PreProcessor to generate unique IDs at runtime:

```
// Withdraw PreProcessor (Groovy)
vars.put("withdrawpatronID",
org.apache.commons.lang3.RandomStringUtils.randomAlphanumeric(10))
vars.put("withdrawtransactionID",
org.apache.commons.lang3.RandomStringUtils.randomAlphanumeric(10))

// Deposit PreProcessor (Groovy)
vars.put("patronID",
org.apache.commons.lang3.RandomStringUtils.randomAlphanumeric(10))
vars.put("transactionID",
org.apache.commons.lang3.RandomStringUtils.randomAlphanumeric(10))
```

Each virtual user generates its own unique IDs per loop, ensuring no two concurrent threads share a TransactionId.

4.7 Response Listeners

Two result collectors are configured in the script:

- **View Results Tree** — Shows individual request/response detail. Useful for debugging. Disable this listener during large load runs to reduce memory usage.
- **Summary Report** — Aggregates throughput, average response time, error rate, and percentiles. Leave this enabled for all runs.

5. Before You Run: Required Configuration

5.1 Update the Bearer Token

The operator JWT token embedded in the script has an expiration date. If the token is expired, all requests will return 401 Unauthorized. Update it before running:

12. Open JMeter GUI and load TransactionFlows.jmx.
13. In the Test Plan tree, expand **Thread Group** → **API Controller** and locate the **User Defined Variables** element at the top of the Test Plan.
14. Find the `token` variable and replace its value with a valid JWT bearer token for the QA operator.
15. Alternatively, update the token in the **HTTP Header Manager** inside each request's Authorization header directly.

NOTE: Obtain a fresh QA operator token from the team's shared credential store or by generating one via the API authentication endpoint. Tokens typically expire after 1 year.

5.2 Verify Target Environment

The script targets QA by default (`qa.api-gaming.paviliononline.io`). To run against a different environment, update the `host` variable in User Defined Variables. Common values:

Environment	Host Value
QA	qa.api-gaming.paviliononline.io
Cert / UAT	cert.api-gaming.paviliononline.io
Production	api-gaming.paviliononline.io (smoke only — do not run load tests)

6. Running the Script — GUI Mode

GUI mode is best for script development, debugging, and small verification runs. Avoid running large load tests (high thread counts) in GUI mode as it consumes significant memory.

16. Open JMeter by double-clicking `jmeter.bat` in the `bin` folder, or run from a command prompt:

```
cd C:\Users\BrockPivirotto\Documents\apache-jmeter-5.5\bin
jmeter.bat
```

17. In the JMeter GUI, click **File** → **Open** and navigate to:

```
C:\Users\BrockPivirotto\OneDrive - Pavilion Payments\Tools\apache-jmeter-5.5\LoadTestScripts\TransactionFlows.jmx
```

18. The test plan tree will appear on the left. Review the Thread Group settings (threads, loops) and confirm the token is valid.
19. Click the green **Play button** (▶) in the toolbar, or use **Run → Start** from the menu.
20. Watch results in real time by clicking **View Results Tree** in the test plan tree. Green results are passing (2xx); red are failures.
21. Click the **Stop button** (■) to stop the test at any time.

NOTE: Clear previous results before each run: click the broom icon (Clear All) in the toolbar. Stale results from prior runs will otherwise accumulate in the Summary Report.

7. Running the Script — Command Line (Non-GUI) Mode

Non-GUI mode is required for CI/CD integration and for any serious load run. It uses far less memory than the GUI and produces a results file you can review afterward.

7.1 Basic Non-GUI Run

Open a command prompt and run:

```
cd C:\Users\BrockPivirotto\Documents\apache-jmeter-5.5\bin

jmeter -n -t "C:\Users\BrockPivirotto\OneDrive - Pavilion Payments\Tools\apache-jmeter-5.5\LoadTestScripts\TransactionFlows.jmx" -l results.jtl -e -o report_output
```

Flag	Meaning
-n	Non-GUI mode (headless)
-t <file>	Path to the .jmx test plan file
-l <file>	Output file for raw results (.jtl format, CSV)
-e	Generate an HTML report after the run
-o <dir>	Output directory for the HTML report (must not already exist)

7.2 Overriding Variables from the Command Line

You can override any test plan variable at runtime using **-J** flags. This is useful for changing the token or thread count without editing the .jmx file:

```
jmeter -n -t TransactionFlows.jmx -l results.jtl ^
```

```
-Jhost=qa.api-gaming.paviliononline.io ^  
-Jtoken=<your_fresh_token> ^  
-l results.jtl
```

7.3 Viewing the HTML Report

After the run completes, open `report_output\index.html` in a browser to see the full HTML dashboard with throughput graphs, response time percentiles, and error analysis.

NOTE: The `-o` output directory must not already exist before the run. Delete or rename it if re-running: `rmdir /s /q report_output`

8. Reading and Interpreting Results

8.1 Summary Report (GUI)

The Summary Report table shows aggregated metrics for each sampler and for the overall test run:

Column	What It Means
# Samples	Total number of HTTP requests sent
Average (ms)	Mean response time across all samples
Min / Max (ms)	Fastest and slowest response times observed
90th %ile (ms)	90% of requests responded within this time — key SLA metric
Error %	Percentage of requests that returned a non-2xx response
Throughput	Requests per second the API served during the test

8.2 Acceptable Performance Thresholds

Use these as reference targets for VIPC API performance:

Metric	Target	Investigate If
Average response time	< 1,000 ms	> 2,000 ms
90th percentile	< 2,000 ms	> 5,000 ms
Error rate	< 0.5%	> 1%
Throughput	Baseline established per run	Drops > 20% from baseline

8.3 Common Errors and Causes

Error / HTTP Code	Likely Cause	Fix
401 Unauthorized	Bearer token expired or wrong operator	Update the token variable in the script
400 Bad Request	Invalid request body (wrong patron data, bad routing/account)	Verify patron VIP card and bank details in the script
Connection refused	VPN not connected, or QA environment is down	Connect to VPN; confirm QA API is up
Timeout	API under heavy load or network latency spike	Reduce thread count; check API server metrics
500 Internal Server Error	Server-side issue, often with duplicate TransactionIds	Confirm Groovy pre-processor is running; check API logs

9. Adjusting Load Parameters

To modify the load profile, open the script in JMeter GUI and update the Thread Group settings:

Scenario	Threads	Loops	Ramp-Up
Smoke / verification	1	1	1s
Light load (dev baseline)	5	10	30s
Standard load (default)	10	100	60s
Stress test	50	100	120s

NOTE: Always coordinate with the dev team before running stress tests against QA. High thread counts can interfere with other QA activities running against the same environment.

10. Available Load Test Scripts

All scripts are stored in: [OneDrive - Pavilion Payments\Tools\apache-jmeter-5.5\LoadTestScripts\](#)

Script File	Purpose	Status
TransactionFlows.jmx	VIPC withdraw + deposit transaction flows — primary script for VIP Preferred SDK load testing	Active (QA)

WithdrawFlow.jmx	Earlier standalone withdraw + deposit script (DEV/GlobalPay endpoints) — reference only	Legacy (DEV)
APITests.jmx	General API endpoint load tests	Review before use
PatronPortal.jmx	Patron portal UI/API load tests	Review before use
TippingTipOutFlow.jmx	Tipping service tip-out flow load tests	Separate product
EIMDepartments.jmx	EIM department API load tests	Separate product
SeleniumSetup.jmx	Selenium integration setup template	Template only
test1.jmx	Ad-hoc scratch script	Review before use

11. Quick Reference

JMeter Location

```
C:\Users\BrockPivirotto\Documents\apache-jmeter-5.5\bin\jmeter.bat
```

VIPC Script Location

```
C:\Users\BrockPivirotto\OneDrive - Pavilion Payments\Tools\apache-jmeter-5.5\LoadTestScripts\TransactionFlows.jmx
```

Start JMeter GUI

```
cd C:\Users\BrockPivirotto\Documents\apache-jmeter-5.5\bin && jmeter.bat
```

Run Headless with HTML Report

```
jmeter -n -t "TransactionFlows.jmx" -l results.jtl -e -o report_output
```

Key Things to Check Before Each Run

- Token is not expired (check exp claim in jwt.io)
- VPN is connected
- QA environment is responding
- Thread count is appropriate for the test goal
- report_output directory does not already exist (for -e -o runs)